Introduction to Programming

Lecture 12

VU
Virtual University

Today's Lecture Includes

- Strings (character arrays)
- Algorithms using arrays
- Multi-dimensional arrays

```
char name [ 100 ] ;
```

\0

In C we have Used

\n New Line

\t Tab Character

\0 Null Character

All C strings are terminated by **Null** character

Character Array in Memory

```
char name [ 100 ] ;
cout << " Please enter your name " ;
cin >> name ;
```

Initializing an Array

Initializing array of integers

```
int c [ 10 ] = { 1,2,3,4,5,6,7,8,9,10 } ;
int c [ ]    = { 1,2,3,4,5,6,7,8,9,10 } ;
```

For character arrays

```
char name [ 100 ] = { 'a',b','c','0','1' } ;
char name [ 100 ] = "abc01" ;
char name [ ] = "Hello World" ;
```

Character Arrays

To read name from keyboard and display it on screen

```
char name [ 100 ] ;
cout << " Please enter you name" ;
cin >> name ;
cout << name ;
```

Character Arrays

Displaying name on screen using loop

```
for ( i = 0 ; i < 100 ; i ++ )
{
 cout << name [ i ] ;
}
```

Comparing Two arrays

Condition :

Array size should be equal

```
int equal = 0 ;
int num1 [ 100 ] , num2 [ 100 ]  ;
for ( i = 0 ; i < 100 ; i ++ )
{
 if ( num1 [ i ] != num2 [ i ] )
 {
  equal = 1 ;
  break ;
 }
}

if ( equal ==1 )
 cout << " The arrays are not equal" ;
else
 cout << " The arrays are equal" ;
```

Comparing Two Arrays

AZMAT HAMEED

Azmat Hameed

Exercise

- Input your name and display it in reverse order
- Determine the length of character array

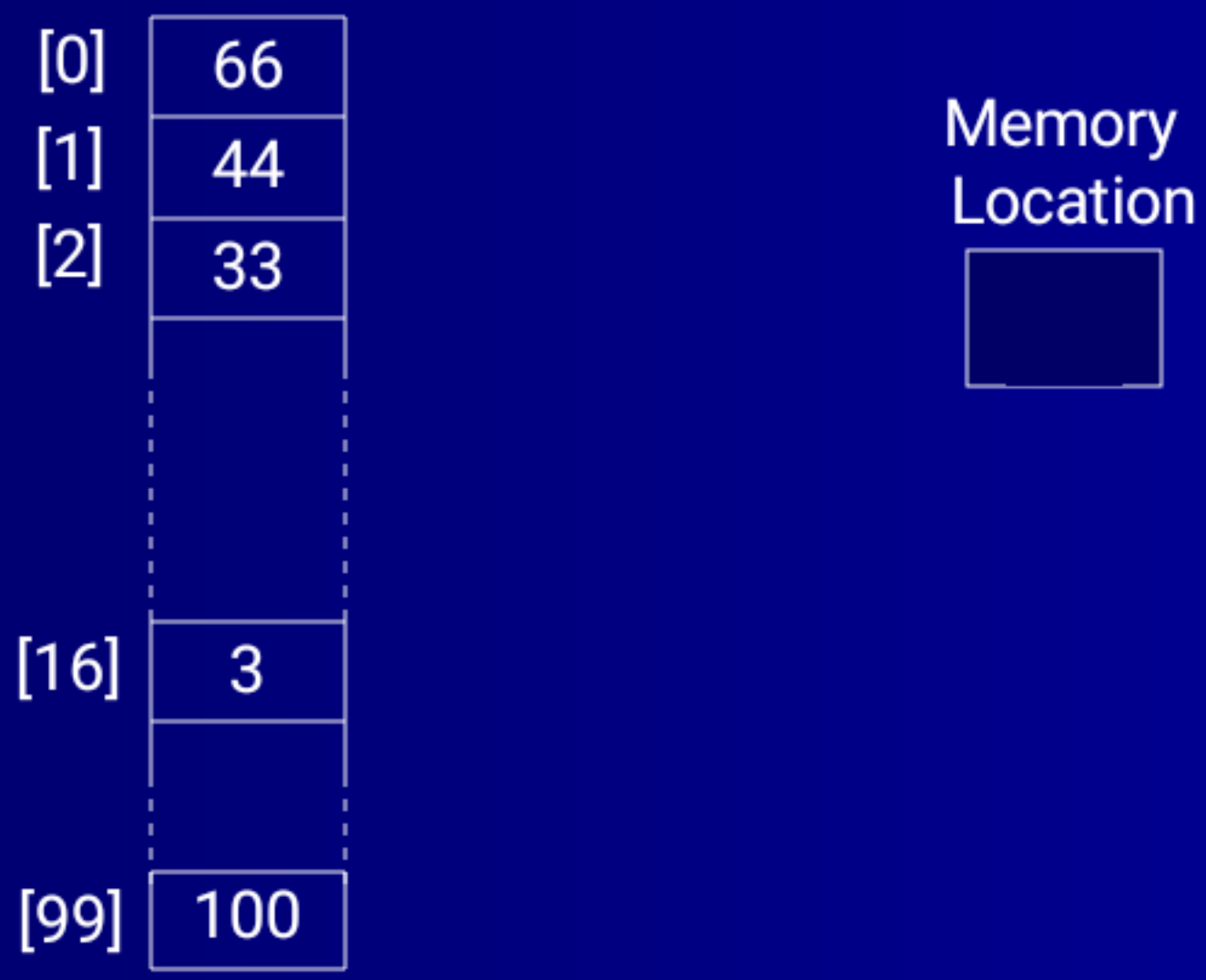
Sorting

- Bubble Sort
- Quick Sort

Brute-Force Technique

[0]	4
[1]	23
[2]	9
[16]	1
[99]	67

Swapping



Swapping Two Numbers

```
int num [ ] ;
int x ;
x = num [ 0 ] ;
num [ 0 ] = num [ 15 ] ;
num [ 15 ] = x ;
```

Binary Search Algorithms

Divide and Conquer rule

Binary Search Algorithm

- If we think about it , it is $\log_n \log_2$ →
- Total array size will be 2^n and number can be found in n times
- If 1000 numbers then 10 tries are max $2^{10} = 1024$

Is divide and conquer the fastest way, all the time, of searching for a number in a list ?

Linear Search ?

Binary Search ?

Suppose they are Random

Suppose they are Ordered

Suppose they are mixed-up

Functions and Arrays

Sending Arrays into Another Functions

- Name of the array
- Size of the array

Example 1

Declaration

```
Char name [ 100 ] ;
```

Function Call

```
reverse ( name , 100 ) ;
```

Example 1

Prototype

```
void reverse ( char [ ] , int ) ;
```

Definition

```
void reverse ( char characters [ ] , int arraySize)
 {
   reverse the character string;
 }
```

Example 1

```
main ( )
 {
   cin >> name [ ] ;
   reverse ( character [ ] , arraySize ) ;
   cout << name [ ] ;
 }
```

What will it Show ?

Call by Reference

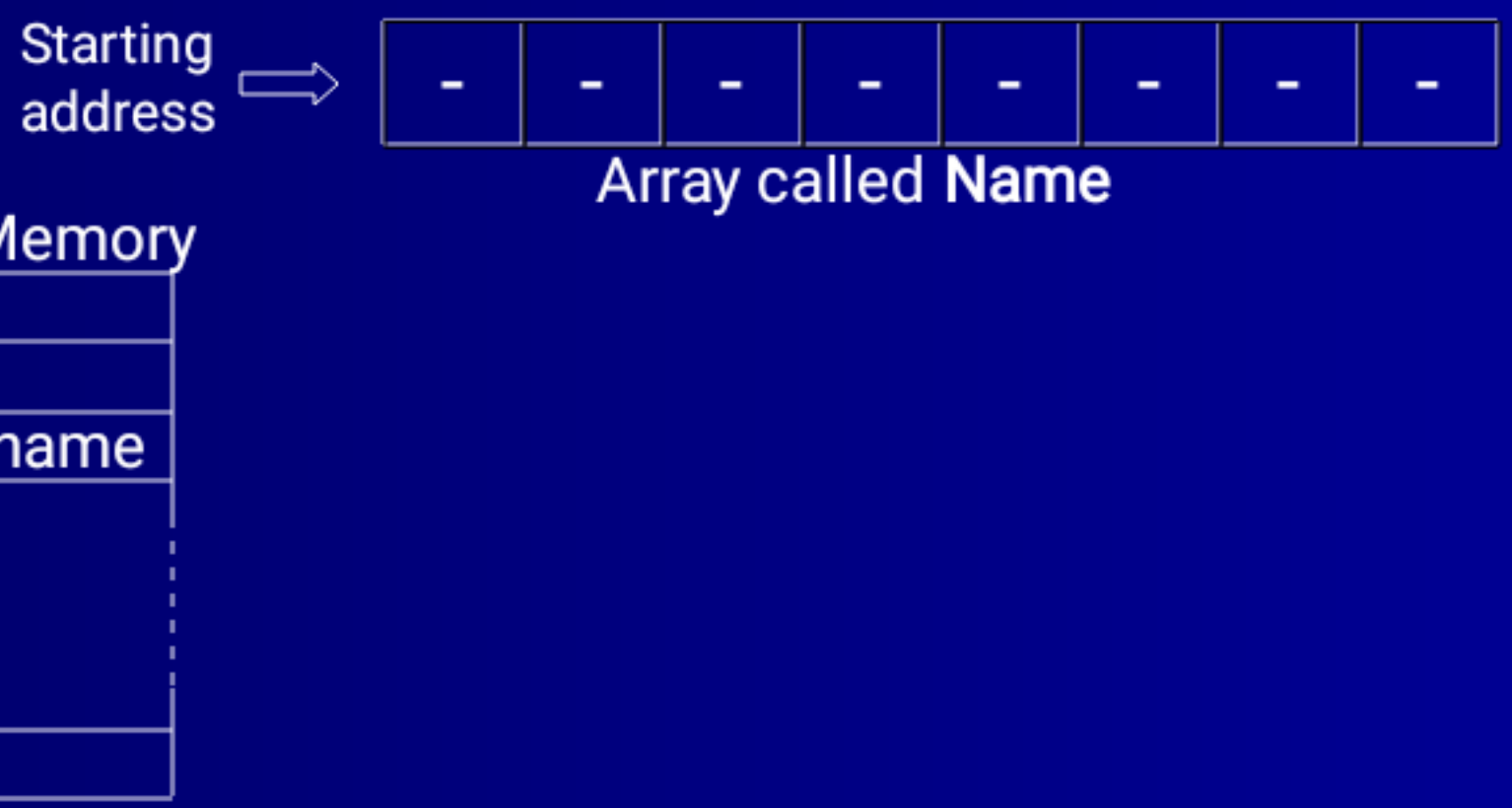
& Address Operator

* Pointer Operator

In case of arrays , call by reference is default

X is a variable which is a location in the memory

Name [ ] is an array



Example 2

```
void f ( int [ ] , int ) ;
main ( )
{
  int numbers [ 100 ] ;
  f ( numbers , 100) ;
  for ( int i = 0 ; i < 100 ; i ++)
   cout << numbers [ i ] ;
}
```

Example 2

```
void f ( int x [ ] , int arraySize )
{
  int i ;
  for ( i = 0 ; i < arraySize ; i ++)
   x [ i ] = i ;
}
```

```
f(x[3]);
```

Executed with call by value, not by reference

- Whenever a variable is passed , it is passed by value
- Whenever you pass an array to function, it is called by reference

Vector

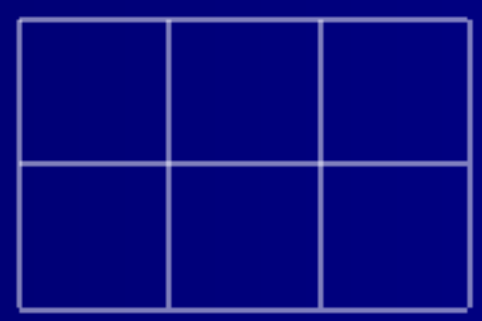
2 Dimensional

3 Dimensional

Dot Product

Vector Product

Matrix



Rows
Columns

Two Dimensional Array

```
int x [ 2 ] [ 3 ] ;
```

Example 3

```
int maxRows = 2;
int maxCols = 3 ;
int matrix [ 2] [ 3 ];
int row , col ;
for ( row = 0 ; row < maxRows ; row ++ )
{
 for ( col = 0 ; col < maxCols ; col ++ )
 {
  cout << "Please enter value of  "<< row << " " << col;
  cin >> matrix [ row ] [ col ] ;
 }
}
```

After first outer loop

[0]

Input

5	2	9

After second outer loop

Input

[0]	5	2	9
[1]	7	0	4

Three Dimensional Arrays

int x [ ] [ ] [ ] ;